

**Name** :- \_Pratiksha Madhav Tonde

**Clg Name** :- T.B.Girwalkr Polytechnic , Ambajogai

**Domain** :- AI - ML

**Branch Name** :- Computer Engineering

<https://github.com/pratikshatonde12-dotcom/Python-Basic-Projects..git>

- **Countries Dataset – Data Reading, Loading, Exploration, and Understanding.**

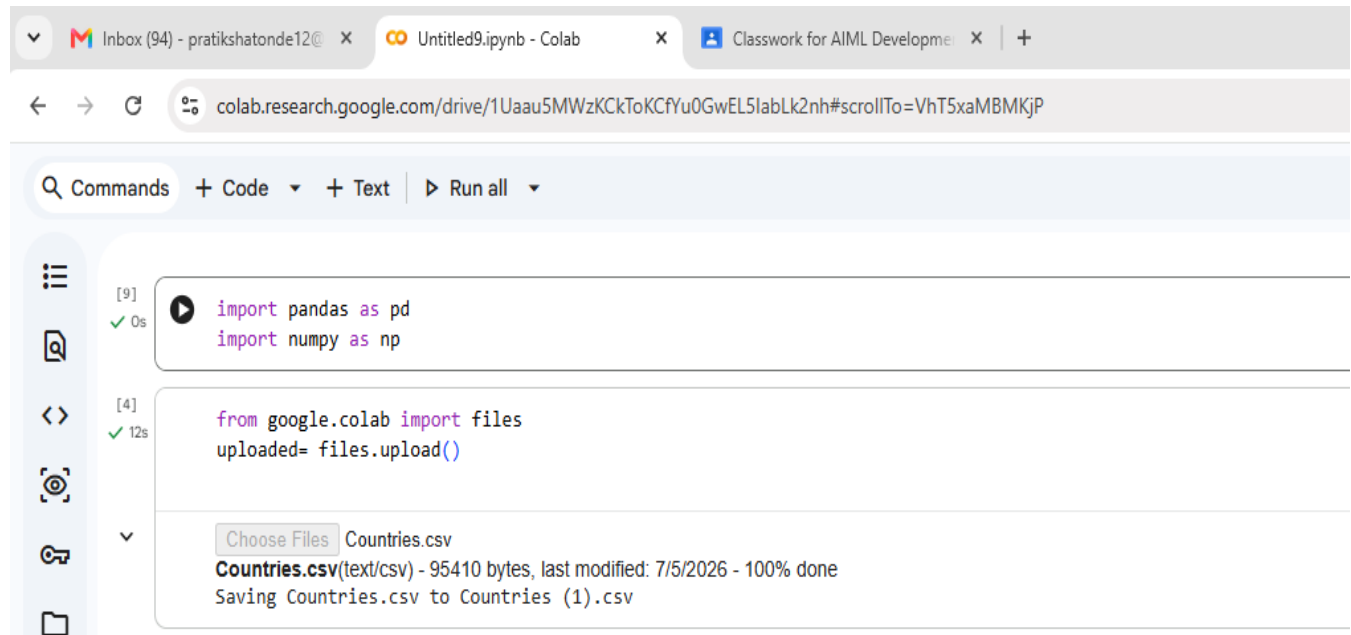
➤ **Aim**

To read, load, explore, and understand the Countries dataset using the Pandas library in Python.

➤ **Objective**

To explain the basic steps of loading a dataset and performing exploratory data analysis (EDA) using commonly used Pandas functions.

# 1) Import the Pandas library.



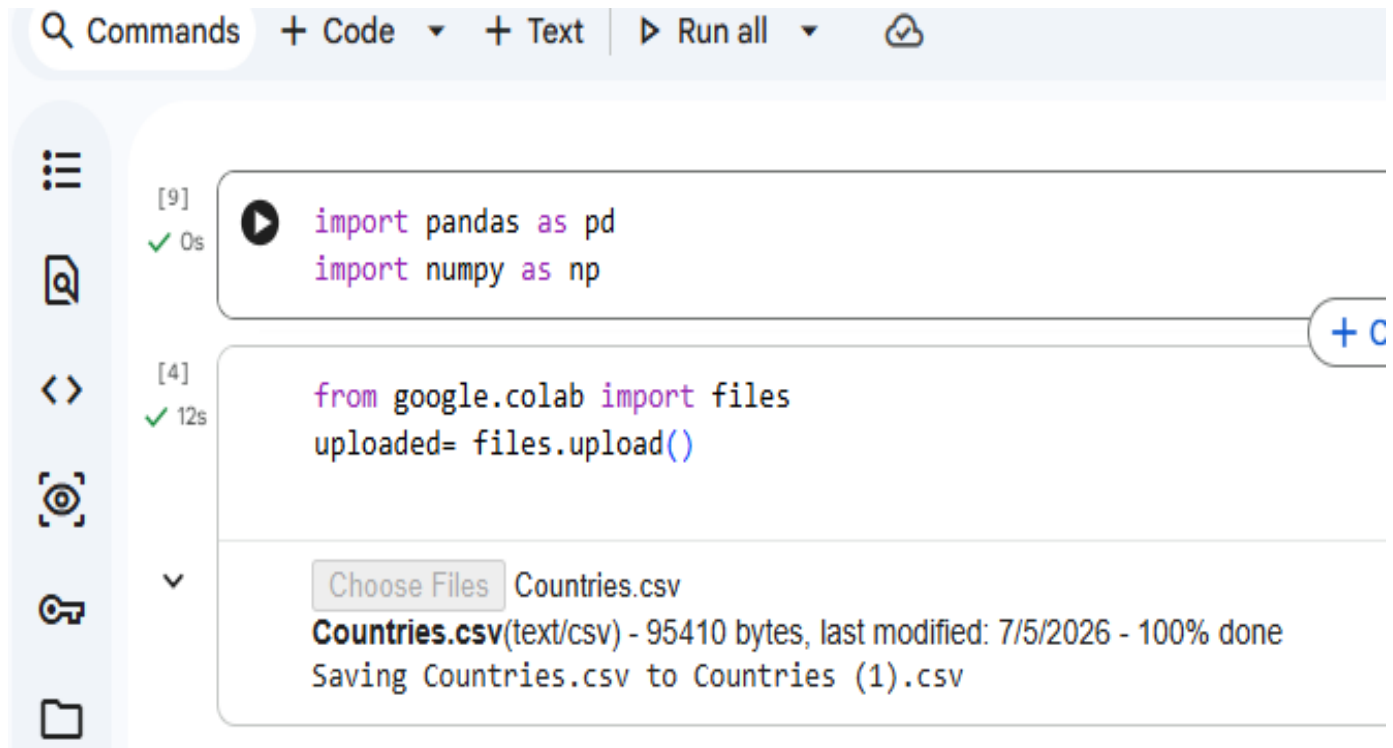
The screenshot shows a Google Colab notebook interface. The browser tabs at the top include 'Inbox (94) - pratikshatonde12@...', 'Untitled9.ipynb - Colab', and 'Classwork for AIML Development...'. The address bar shows the URL 'colab.research.google.com/drive/1Uau5MWzKCKToKCfYu0GwEL5labLk2nh#scrollTo=VhT5xaMBMKjP'. The notebook toolbar shows 'Commands', '+ Code', '+ Text', and 'Run all'. The left sidebar contains icons for file management, search, and other tools. The main code area displays two code cells. The first cell, labeled '[9]', contains the code 'import pandas as pd' and 'import numpy as np', with a green checkmark and '0s' indicating successful execution. The second cell, labeled '[4]', contains the code 'from google.colab import files' and 'uploaded= files.upload()', with a green checkmark and '12s' indicating successful execution. Below the code cells, a file upload dialog is visible, showing a button 'Choose Files' and the filename 'Countries.csv'. Below the filename, it says 'Countries.csv(text/csv) - 95410 bytes, last modified: 7/5/2026 - 100% done' and 'Saving Countries.csv to Countries (1).csv'.

```
[9] import pandas as pd
import numpy as np
```

```
[4] from google.colab import files
uploaded= files.upload()
```

Choose Files Countries.csv  
Countries.csv(text/csv) - 95410 bytes, last modified: 7/5/2026 - 100% done  
Saving Countries.csv to Countries (1).csv

2) Load the CSV dataset into a DataFrame.



The image shows a Google Colab interface. At the top, there is a toolbar with a search icon, the text "Commands", and buttons for "+ Code", "+ Text", "Run all", and a cloud icon. On the left side, there is a vertical sidebar with icons for a menu, a search icon, a code editor, a terminal, a file explorer, and a folder icon. The main area displays two code cells. The first cell, labeled "[9]" with a green checkmark and "0s", contains the code: `import pandas as pd` and `import numpy as np`. The second cell, labeled "[4]" with a green checkmark and "12s", contains the code: `from google.colab import files` and `uploaded= files.upload()`. Below the second cell, there is a file upload section showing a "Choose Files" button, the filename "Countries.csv", and the status: "Countries.csv(text/csv) - 95410 bytes, last modified: 7/5/2026 - 100% done" and "Saving Countries.csv to Countries (1).csv".

Commands + Code + Text Run all

[9] ✓ 0s

```
import pandas as pd
import numpy as np
```

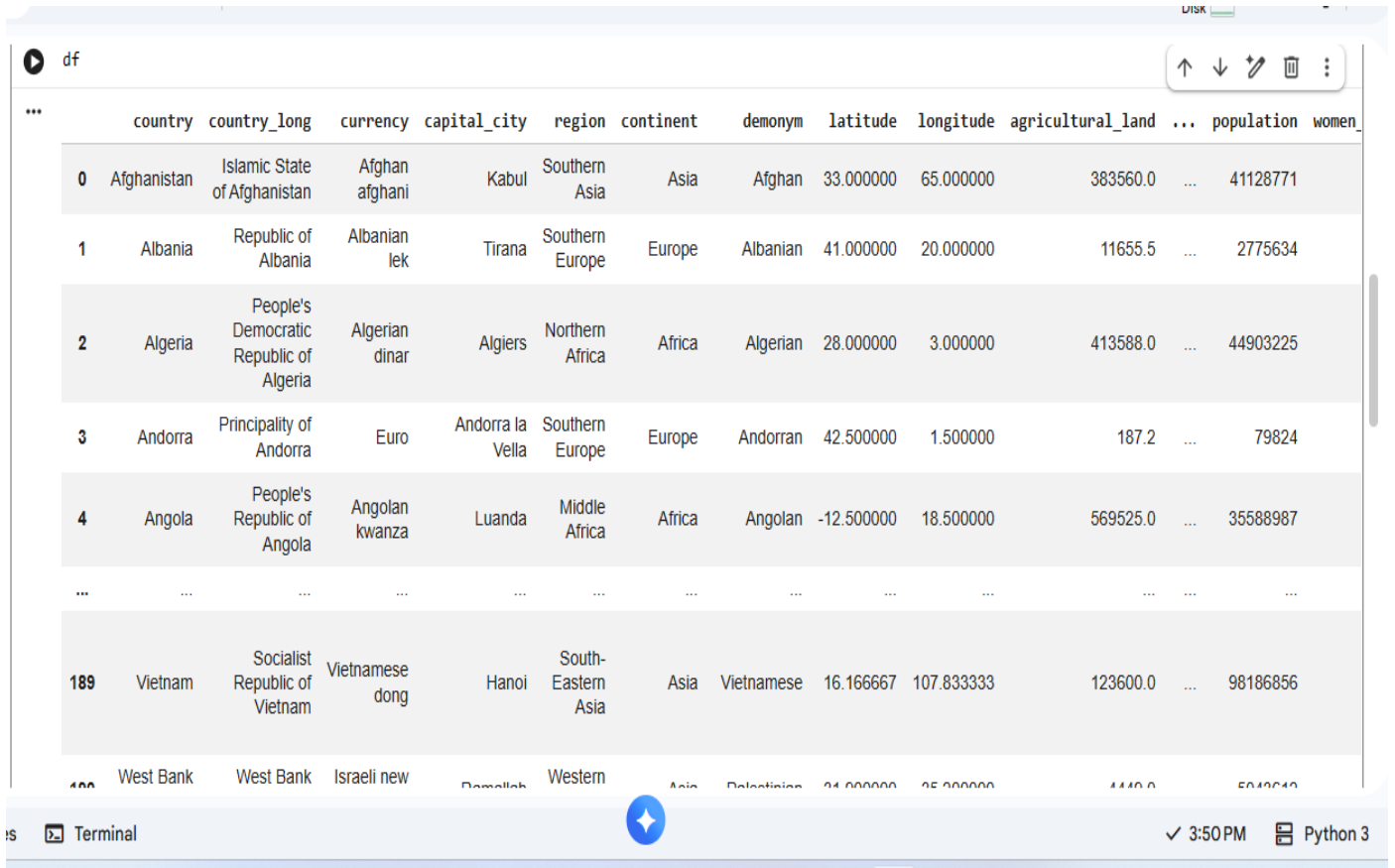
[4] ✓ 12s

```
from google.colab import files
uploaded= files.upload()
```

Choose Files Countries.csv

**Countries.csv**(text/csv) - 95410 bytes, last modified: 7/5/2026 - 100% done  
Saving Countries.csv to Countries (1).csv

### 3) Display the all dataset.



The screenshot shows a Jupyter Notebook interface with a DataFrame named 'df'. The DataFrame contains the following data:

|     | country     | country_long                            | currency        | capital_city     | region             | continent | demonym     | latitude   | longitude  | agricultural_land | ... | population | women |
|-----|-------------|-----------------------------------------|-----------------|------------------|--------------------|-----------|-------------|------------|------------|-------------------|-----|------------|-------|
| 0   | Afghanistan | Islamic State of Afghanistan            | Afghan afghani  | Kabul            | Southern Asia      | Asia      | Afghan      | 33.000000  | 65.000000  | 383560.0          | ... | 41128771   |       |
| 1   | Albania     | Republic of Albania                     | Albanian lek    | Tirana           | Southern Europe    | Europe    | Albanian    | 41.000000  | 20.000000  | 11655.5           | ... | 2775634    |       |
| 2   | Algeria     | People's Democratic Republic of Algeria | Algerian dinar  | Algiers          | Northern Africa    | Africa    | Algerian    | 28.000000  | 3.000000   | 413588.0          | ... | 44903225   |       |
| 3   | Andorra     | Principality of Andorra                 | Euro            | Andorra la Vella | Southern Europe    | Europe    | Andorran    | 42.500000  | 1.500000   | 187.2             | ... | 79824      |       |
| 4   | Angola      | People's Republic of Angola             | Angolan kwanza  | Luanda           | Middle Africa      | Africa    | Angolan     | -12.500000 | 18.500000  | 569525.0          | ... | 35588987   |       |
| ... | ...         | ...                                     | ...             | ...              | ...                | ...       | ...         | ...        | ...        | ...               | ... | ...        | ...   |
| 189 | Vietnam     | Socialist Republic of Vietnam           | Vietnamese dong | Hanoi            | South-Eastern Asia | Asia      | Vietnamese  | 16.166667  | 107.833333 | 123600.0          | ... | 98186856   |       |
| 190 | West Bank   | West Bank                               | Israeli new     | Ramallah         | Western Asia       | Asia      | Palestinian | 31.000000  | 35.000000  | 4440.0            | ... | 5042640    |       |

The interface includes a terminal at the bottom with the text 'Terminal' and a status bar showing '3:50 PM' and 'Python 3'.

After loading the CSV file into a DataFrame, you can display all the data

#### 4) Display the all columns in dataset.

13]  
0s



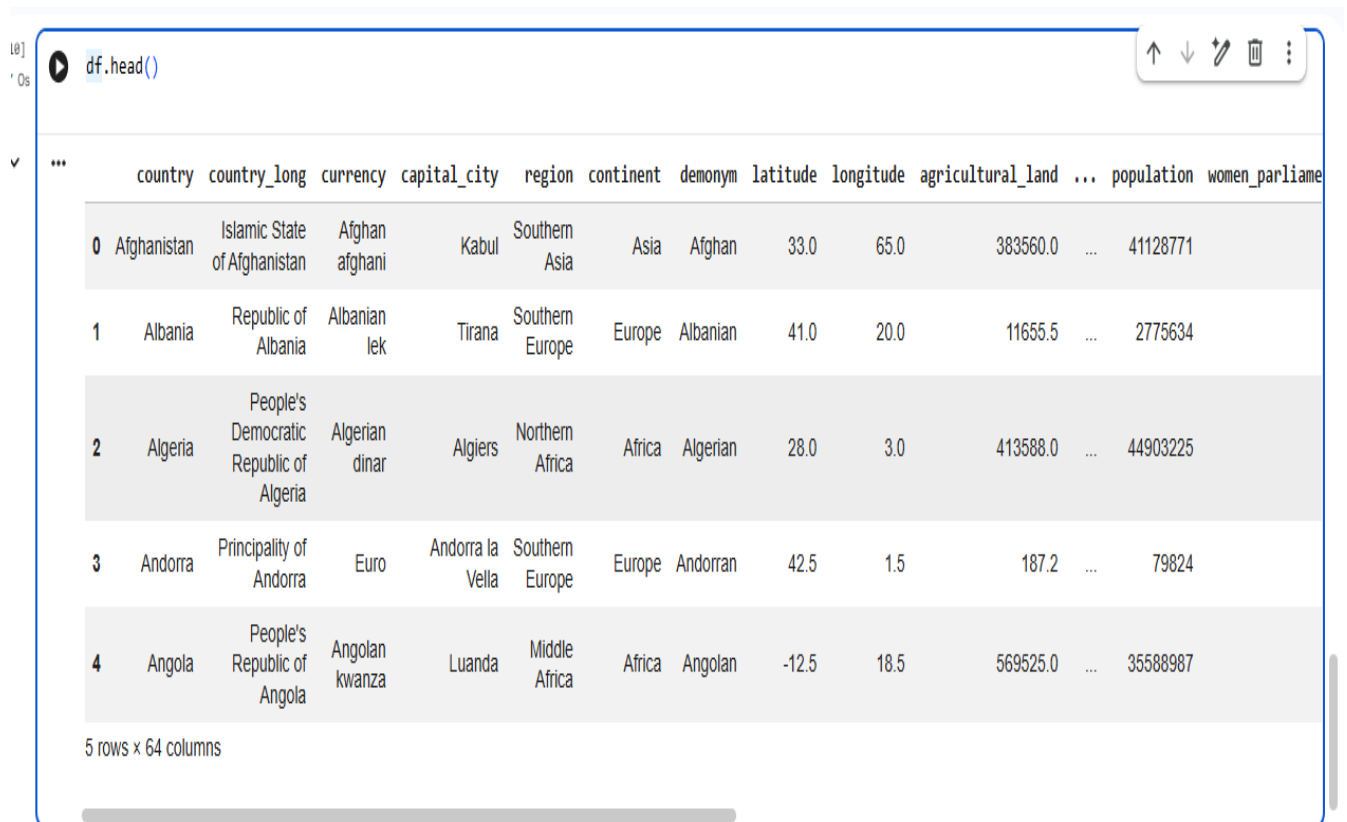
df.columns

✓

```
*** Index(['country', 'country_long', 'currency', 'capital_city', 'region',  
          'continent', 'demonym', 'latitude', 'longitude', 'agricultural_land',  
          'forest_area', 'land_area', 'rural_land', 'urban_land',  
          'central_government_debt_pct_gdp', 'expense_pct_gdp', 'gdp',  
          'inflation', 'self-employed_pct', 'tax_revenue_pct_gdp',  
          'unemployment_pct', 'vulnerable_employment_pct',  
          'electricity_access_pct', 'alternative_nuclear_energy_pct',  
          'electricity_production_coal_pct',  
          'electricity_production_hydroelectric_pct',  
          'electricity_production_gas_pct', 'electricity_production_nuclear_pct',  
          'electricity_production_oil_pct', 'electricity_production_renewable_pct',  
          'energy_imports_pct', 'fossil_energy_consumption_pct',  
          'renewable_energy_consumption_pct', 'co2_emissions',  
          'methane_emissions', 'nitrous_oxide_emissions',  
          'greenhouse_other_emissions', 'urban_population_under_5m',  
          'health_expenditure_pct_gdp', 'health_expenditure_capita',  
          'hospital_beds', 'hiv_incidence', 'suicide_rate', 'armed_forces',  
          'internally_displaced_persons', 'military_expenditure_pct_gdp',  
          'birth_rate', 'death_rate', 'fertility_rate', 'internet_pct',  
          'life_expectancy', 'net_migration', 'population_female',  
          'population_male', 'population', 'women_parliament_seats_pct',  
          'rural_population', 'urban_population', 'press', 'democracy_score',  
          'democracy_type', 'median_age', 'political_leader', 'title'],  
          dtype='object')
```

To display the names of all columns in a Pandas DataFrame, use the `columns` attribute

1) Display the first 5 rows of the dataset.



The screenshot shows a Jupyter Notebook interface. The code cell contains `df.head()`. The output is a Pandas DataFrame with 5 rows and 14 columns. The columns are: country, country\_long, currency, capital\_city, region, continent, demonym, latitude, longitude, agricultural\_land, ..., population, women\_parliame. The rows represent Afghanistan, Albania, Algeria, Andorra, and Angola.

|   | country     | country_long                            | currency       | capital_city     | region          | continent | demonym  | latitude | longitude | agricultural_land | ... | population | women_parliame |
|---|-------------|-----------------------------------------|----------------|------------------|-----------------|-----------|----------|----------|-----------|-------------------|-----|------------|----------------|
| 0 | Afghanistan | Islamic State of Afghanistan            | Afghan afghani | Kabul            | Southern Asia   | Asia      | Afghan   | 33.0     | 65.0      | 383560.0          | ... | 41128771   |                |
| 1 | Albania     | Republic of Albania                     | Albanian lek   | Tirana           | Southern Europe | Europe    | Albanian | 41.0     | 20.0      | 11655.5           | ... | 2775634    |                |
| 2 | Algeria     | People's Democratic Republic of Algeria | Algerian dinar | Algiers          | Northern Africa | Africa    | Algerian | 28.0     | 3.0       | 413588.0          | ... | 44903225   |                |
| 3 | Andorra     | Principality of Andorra                 | Euro           | Andorra la Vella | Southern Europe | Europe    | Andorran | 42.5     | 1.5       | 187.2             | ... | 79824      |                |
| 4 | Angola      | People's Republic of Angola             | Angolan kwanza | Luanda           | Middle Africa   | Africa    | Angolan  | -12.5    | 18.5      | 569525.0          | ... | 35588987   |                |

5 rows x 64 columns

To display the first **5 rows** of a Pandas DataFrame, use the `head()` function

## 2) Display the last 5 rows of the dataset.

df.tail()

|     | country            | country_long                  | currency           | capital_city | region             | continent | demonym     | latitude   | longitude  | agricultural_land | ... | population | women_p |
|-----|--------------------|-------------------------------|--------------------|--------------|--------------------|-----------|-------------|------------|------------|-------------------|-----|------------|---------|
| 189 | Vietnam            | Socialist Republic of Vietnam | Vietnamese dong    | Hanoi        | South-Eastern Asia | Asia      | Vietnamese  | 16.166667  | 107.833333 | 123600.0          | ... | 98186856   |         |
| 190 | West Bank and Gaza | West Bank and Gaza            | Israeli new shekel | Ramallah     | Western Asia       | Asia      | Palestinian | 31.900000  | 35.200000  | 4449.0            | ... | 5043612    |         |
| 191 | Yemen              | Republic of Yemen             | Yemeni rial        | Sana'a       | Western Asia       | Asia      | Yemeni      | 15.000000  | 48.000000  | 234520.0          | ... | 33696614   |         |
| 192 | Zambia             | Republic of Zambia            | New Zambian kwacha | Lusaka       | Eastern Africa     | Africa    | Zambian     | -15.000000 | 30.000000  | 238360.0          | ... | 20017675   |         |
| 193 | Zimbabwe           | Republic of Zimbabwe          | U.S. dollar        | Harare       | Eastern Africa     | Africa    | Zimbabwean  | -20.000000 | 30.000000  | 162000.0          | ... | 16320537   |         |

5 rows x 64 columns

To display the last **5 rows** of a Pandas DataFrame, use the `tail()` function

3) Show the number of rows and columns.

```
[17]  
✓ Os df.shape  
▼ ... (194, 64)
```

Using shape display rows and columns



4) Show dataset information, data types, and non-null values.

```
[18] ✓ 0s df.info()
...
8 longitude 194 non-null float64
9 agricultural_land 193 non-null float64
10 forest_area 194 non-null float64
11 land_area 194 non-null float64
12 rural_land 194 non-null float64
13 urban_land 194 non-null float64
14 central_government_debt_pct_gdp 120 non-null float64
15 expense_pct_gdp 156 non-null float64
16 gdp 193 non-null float64
17 inflation 184 non-null float64
18 self_employed_pct 179 non-null float64
19 tax_revenue_pct_gdp 159 non-null float64
20 unemployment_pct 179 non-null float64
21 vulnerable_employment_pct 179 non-null float64
22 electricity_access_pct 194 non-null float64
23 alternative_nuclear_energy_pct 138 non-null float64
24 electricity_production_coal_pct 138 non-null float64
25 electricity_production_hydroelectric_pct 138 non-null float64
26 electricity_production_gas_pct 138 non-null float64
27 electricity_production_nuclear_pct 138 non-null float64
28 electricity_production_oil_pct 138 non-null float64
29 electricity_production_renewable_pct 138 non-null float64
30 energy_imports_pct 138 non-null float64
31 fossil_energy_consumption_pct 168 non-null float64
32 renewable_energy_consumption_pct 192 non-null float64
33 co2_emissions 191 non-null float64
34 methane_emissions 191 non-null float64
35 nitrous_oxide_emissions 191 non-null float64
```

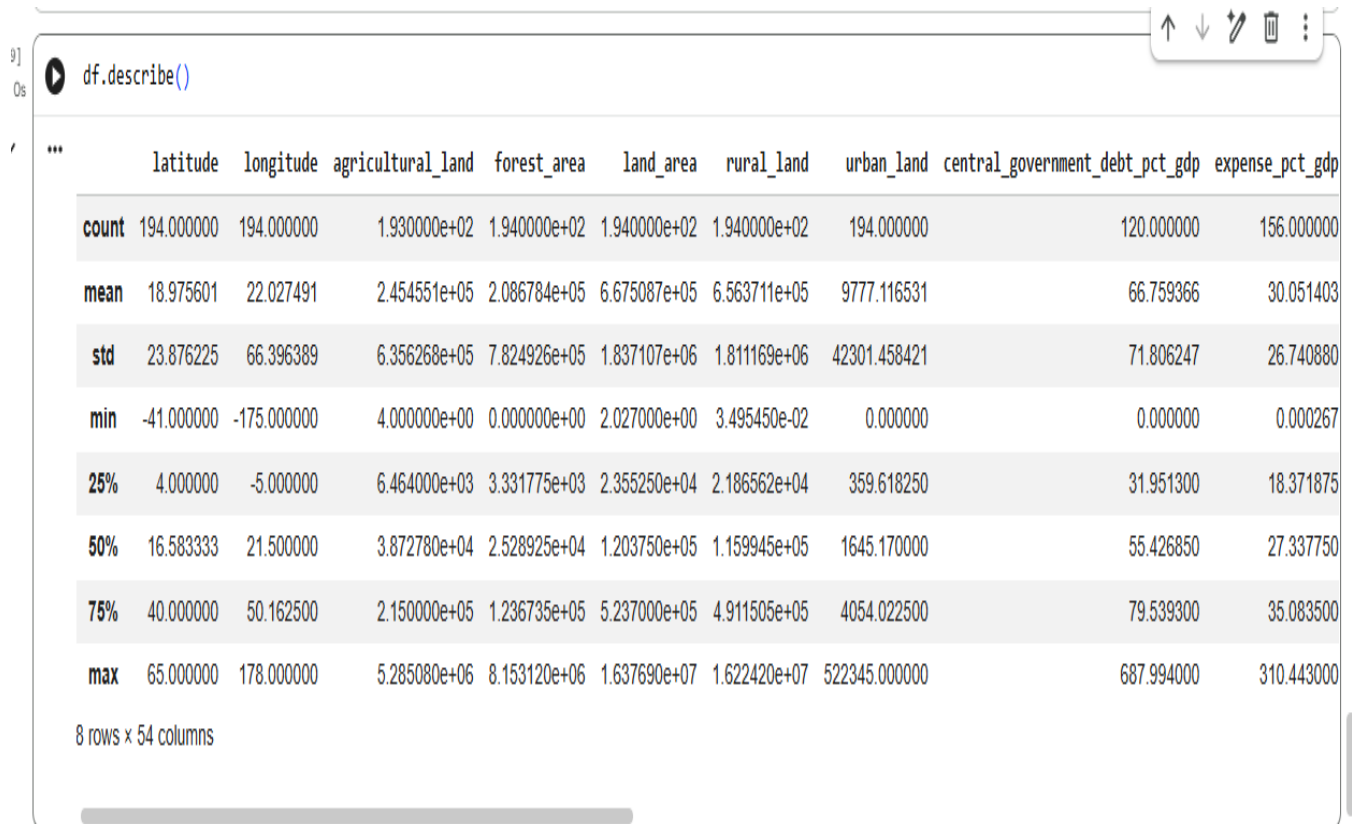
```

... 36 greenhouse_other_emissions      184 non-null    float64
    37 urban_population_under_5m      194 non-null    float64
    38 health_expenditure_pct_gdp     190 non-null    float64
    39 health_expenditure_capita      190 non-null    float64
    40 hospital_beds                  192 non-null    float64
    41 hiv_incidence                  133 non-null    float64
    42 suicide_rate                   183 non-null    float64
    43 armed_forces                   174 non-null    float64
    44 internally_displaced_persons     73 non-null     float64
    45 military_expenditure_pct_gdp    163 non-null    float64
    46 birth_rate                     194 non-null    float64
    47 death_rate                     194 non-null    float64
    48 fertility_rate                 193 non-null    float64
    49 internet_pct                   194 non-null    float64
    50 life_expectancy                191 non-null    float64
    51 net_migration                   194 non-null    int64
    52 population_female              194 non-null    int64
    53 population_male                194 non-null    int64
    54 population                     194 non-null    int64
    55 women_parliament_seats_pct      193 non-null    float64
    56 rural_population               194 non-null    int64
    57 urban_population               194 non-null    int64
    58 press                          194 non-null    float64
    59 democracy_score                 194 non-null    float64
    60 democracy_type                 194 non-null    object
    61 median_age                     194 non-null    float64
    62 political_leader                187 non-null    object
    63 title                          187 non-null    object
dtypes: float64(48), int64(6), object(10)
memory usage: 97.1+ KB

```

To display information about a Pandas DataFrame, including **column names, data types, and non-null values**, use the `info()` function

## 5) Generate summary statistics for numerical columns.



The image shows a Jupyter Notebook cell with the code `df.describe()` and its output. The output is a summary statistics table for numerical columns. The table has 10 columns: `latitude`, `longitude`, `agricultural_land`, `forest_area`, `land_area`, `rural_land`, `urban_land`, `central_government_debt_pct_gdp`, and `expense_pct_gdp`. The rows represent different statistical measures: `count`, `mean`, `std`, `min`, `25%`, `50%`, `75%`, and `max`. The output is displayed in a table format with alternating light and dark gray rows. Below the table, it indicates '8 rows x 54 columns'.

|       | latitude   | longitude   | agricultural_land | forest_area  | land_area    | rural_land   | urban_land    | central_government_debt_pct_gdp | expense_pct_gdp |
|-------|------------|-------------|-------------------|--------------|--------------|--------------|---------------|---------------------------------|-----------------|
| count | 194.000000 | 194.000000  | 1.930000e+02      | 1.940000e+02 | 1.940000e+02 | 1.940000e+02 | 194.000000    | 120.000000                      | 156.000000      |
| mean  | 18.975601  | 22.027491   | 2.454551e+05      | 2.086784e+05 | 6.675087e+05 | 6.563711e+05 | 9777.116531   | 66.759366                       | 30.051403       |
| std   | 23.876225  | 66.396389   | 6.356268e+05      | 7.824926e+05 | 1.837107e+06 | 1.811169e+06 | 42301.458421  | 71.806247                       | 26.740880       |
| min   | -41.000000 | -175.000000 | 4.000000e+00      | 0.000000e+00 | 2.027000e+00 | 3.495450e-02 | 0.000000      | 0.000000                        | 0.000267        |
| 25%   | 4.000000   | -5.000000   | 6.464000e+03      | 3.331775e+03 | 2.355250e+04 | 2.186562e+04 | 359.618250    | 31.951300                       | 18.371875       |
| 50%   | 16.583333  | 21.500000   | 3.872780e+04      | 2.528925e+04 | 1.203750e+05 | 1.159945e+05 | 1645.170000   | 55.426850                       | 27.337750       |
| 75%   | 40.000000  | 50.162500   | 2.150000e+05      | 1.236735e+05 | 5.237000e+05 | 4.911505e+05 | 4054.022500   | 79.539300                       | 35.083500       |
| max   | 65.000000  | 178.000000  | 5.285080e+06      | 8.153120e+06 | 1.637690e+07 | 1.622420e+07 | 522345.000000 | 687.994000                      | 310.443000      |

8 rows x 54 columns

To generate **summary statistics** for all numerical columns in a Pandas DataFrame, use the `describe()` function.

6) Display the data type of each column.

```
[20] ✓ 0s
df.dtypes
```

|                  | 0       |
|------------------|---------|
| country          | object  |
| country_long     | object  |
| currency         | object  |
| capital_city     | object  |
| region           | object  |
| ...              | ...     |
| democracy_score  | float64 |
| democracy_type   | object  |
| median_age       | float64 |
| political_leader | object  |
| title            | object  |

64 rows × 1 columns

dtype: object

To display the **data type of each column** in a Pandas DataFrame, use the `dtypes` attribute

7) Count missing values in each column.

```
[21]
✓ Os
df.isnull().sum()
```

|                  |     |
|------------------|-----|
| ...              | 0   |
| country          | 0   |
| country_long     | 0   |
| currency         | 0   |
| capital_city     | 0   |
| region           | 0   |
| ...              | ... |
| democracy_score  | 0   |
| democracy_type   | 0   |
| median_age       | 0   |
| political_leader | 7   |
| title            | 7   |

64 rows × 1 columns

dtype: int64

To count the number of **missing (null) values** in each column of a Pandas DataFrame, use the `isnull()` and `sum()` functions.

8) Count unique values in each column.

[22] ✓ 0s

df.nunique()

|                  |     |
|------------------|-----|
| ...              | 0   |
| country          | 194 |
| country_long     | 194 |
| currency         | 142 |
| capital_city     | 194 |
| region           | 22  |
| ...              | ... |
| democracy_score  | 146 |
| democracy_type   | 5   |
| median_age       | 143 |
| political_leader | 187 |
| title            | 17  |

64 rows × 1 columns

dtype: int64

To count the number of **unique (distinct) values** in each column of a Pandas DataFrame, use the `nunique()` function

## 9) Display 5 random rows from the dataset.

[23] ✓ 0s df.sample(5)

|     | country      | country_long            | currency                  | capital_city | region          | continent | demonym     | latitude  | longitude  | agricultural_land | ... | population | women_pa |
|-----|--------------|-------------------------|---------------------------|--------------|-----------------|-----------|-------------|-----------|------------|-------------------|-----|------------|----------|
| 38  | Costa Rica   | Republic of Costa Rica  | Costa Rican colon         | San José     | Central America | Americas  | Costa Rican | 10.000000 | -84.000000 | 17615.0           | ... | 5180829    |          |
| 178 | Turkmenistan | Turkmenistan            | New Turkmen manat         | Ashgabat     | Central Asia    | Asia      | Turkmen     | 40.000000 | 60.000000  | 338380.0          | ... | 6430770    |          |
| 52  | El Salvador  | Republic of El Salvador | U.S. dollar               | San Salvador | Central America | Americas  | Salvadoran  | 13.833333 | -88.916667 | 11957.0           | ... | 6336392    |          |
| 61  | Gabon        | Gabonese Republic       | Central African CFA franc | Libreville   | Middle Africa   | Africa    | Gabonese    | -1.000000 | 11.750000  | 22126.4           | ... | 2388992    |          |
| 1   | Albania      | Republic of Albania     | Albanian lek              | Tirana       | Southern Europe | Europe    | Albanian    | 41.000000 | 20.000000  | 11655.5           | ... | 2775634    |          |

5 rows x 64 columns

## 10) Count occurrences of unique values (for a selected column).

df.value\_counts()

| country    | country_long                    | currency                  | capital_city | region        | continent | demonym     | latitude   | longitude  | agricultural_land | forest_area | land_area |
|------------|---------------------------------|---------------------------|--------------|---------------|-----------|-------------|------------|------------|-------------------|-------------|-----------|
| Azerbaijan | Republic of Azerbaijan          | New Azeri manat           | Baku         | Western Asia  | Asia      | Azerbaijani | 40.500000  | 47.500000  | 47801.00          | 11317.7     | 82646.0   |
| Bangladesh | People's Republic of Bangladesh | Bangladeshi taka          | Dhaka        | Southern Asia | Asia      | Bangladeshi | 24.000000  | 90.000000  | 99010.00          | 18834.0     | 130170.0  |
| Bolivia    | Plurinational State of Bolivia  | Bolivian Boliviano        | Sucre        | South America | Americas  | Bolivian    | -17.000000 | -65.000000 | 377870.00         | 508338.0    | 1083300   |
| Brazil     | Federative Republic of Brazil   | Brazilian real            | Brasília     | South America | Americas  | Brazilian   | -10.000000 | -55.000000 | 2368790.00        | 4966200.0   | 8358140   |
| Cameroon   | Republic of Cameroon            | Central African CFA franc | Yaoundé      | Middle Africa | Africa    | Cameroonian | 6.000000   | 12.000000  | 97500.00          | 203405.0    | 472710.0  |
| Colombia   | Republic of Colombia            | Colombian peso            | Bogotá       | South America | Americas  | Colombian   | 4.000000   | -72.000000 | 482428.00         | 591419.0    | 1109500   |
| Congo      | Republic of Congo               | Central African CFA franc | Brazzaville  | Middle Africa | Africa    | Congolese   | -1.000000  | 15.000000  | 106280.00         | 219460.0    | 341500.0  |

Variables Terminal 4:19 PM Python 3

to count how many times each unique value appears in a selected column, use the `value_counts()` function



|     |                 |                                         |                        |                |                 |          |            |           |            |            |           |         |
|-----|-----------------|-----------------------------------------|------------------------|----------------|-----------------|----------|------------|-----------|------------|------------|-----------|---------|
| ... | Croatia         | Republic of Croatia                     | Croatian kuna          | Zagreb         | Southern Europe | Europe   | Croatian   | 45.166667 | 15.500000  | 15050.00   |           |         |
|     | Cyprus          | Republic of Cyprus                      | Euro                   | Nicosia        | Southern Europe | Europe   | Cypriot    | 35.000000 | 33.000000  | 1341.29    | 1725.3    | 9240.0  |
|     | Côte d'Ivoire   | Republic of Côte d'Ivoire               | West African CFA franc | Yamoussoukro   | Western Africa  | Africa   | Ivorian    | 8.000000  | -5.000000  | 212000.00  | 28367.1   | 318000. |
|     | Dem. Rep. Congo | Democratic Republic of the Congo        | Congolese franc        | Kinshasa       | Middle Africa   | Africa   | Congolese  | 0.000000  | 25.000000  | 335720.00  | 1261550.0 | 2267050 |
|     | El Salvador     | Republic of El Salvador                 | U.S. dollar            | San Salvador   | Central America | Americas | Salvadoran | 13.833333 | -88.916667 | 11957.00   | 5838.8    | 20720.0 |
|     | Ethiopia        | Federal Democratic Republic of Ethiopia | Ethiopian birr         | Addis Ababa    | Eastern Africa  | Africa   | Ethiopian  | 8.000000  | 38.000000  | 384761.00  | 170685.0  | 1128570 |
|     | Georgia         | Georgia                                 | Georgian lari          | Tbilisi        | Western Asia    | Asia     | Georgian   | 42.000000 | 43.500000  | 23779.00   | 28224.0   | 69490.0 |
|     | Guatemala       | Republic of Guatemala                   | Guatemalan quetzal     | Guatemala City | Central America | Americas | Guatemalan | 15.500000 | -90.250000 | 38560.00   | 35278.0   | 107160. |
|     | India           | Republic of India                       | Indian rupee           | New Delhi      | Southern Asia   | Asia     | Indian     | 20.000000 | 77.000000  | 1790450.00 | 721600.0  | 2973190 |

|     |                 |                                  |                   |             |                    |          |             |            |             |            |          |         |
|-----|-----------------|----------------------------------|-------------------|-------------|--------------------|----------|-------------|------------|-------------|------------|----------|---------|
| ... | Indonesia       | Republic of Indonesia            | Indonesian rupiah | Jakarta     | South-Eastern Asia | Asia     | Indonesian  | -5.000000  | 120.000000  | 623000.00  |          |         |
|     | Kazakhstan      | Republic of Kazakhstan           | Kazakh tenge      | Astana      | Central Asia       | Asia     | Kazakhstani | 48.000000  | 68.000000   | 2140030.00 | 34546.8  | 2699700 |
|     | Kyrgyz Republic | Kyrgyz Republic                  | Kyrgyz som        | Bishkek     | Central Asia       | Asia     | Kirghiz     | 41.000000  | 75.000000   | 103678.00  | 13153.8  | 191800. |
|     | Lebanon         | Lebanese Republic                | Lebanese pound    | Beirut      | Western Asia       | Asia     | Lebanese    | 33.833333  | 35.833333   | 6696.70    | 1433.3   | 10230.0 |
|     | Mexico          | United Mexican States            | Mexican peso      | Mexico City | Central America    | Americas | Mexican     | 23.000000  | -102.000000 | 971380.00  | 656921.0 | 1943950 |
|     | Myanmar         | Republic of the Union of Myanmar | Myanmar kyat      | Naypyidaw   | South-Eastern Asia | Asia     | Burmese     | 22.000000  | 98.000000   | 130080.00  | 285439.0 | 652670. |
|     | Peru            | Republic of Peru                 | Peruvian new sol  | Lima        | South America      | Americas | Peruvian    | -10.000000 | -76.000000  | 244783.00  | 723304.0 | 1280000 |

...

|             |                                            |                   |          |                    |        |            |           |            |           |          |          |
|-------------|--------------------------------------------|-------------------|----------|--------------------|--------|------------|-----------|------------|-----------|----------|----------|
| Philippines | Republic of the Philippines                | Philippine peso   | Manila   | South-Eastern Asia | Asia   | Filipino   | 13.000000 | 122.000000 | 126750.00 |          |          |
| Sri Lanka   | Democratic Socialist Republic of Sri Lanka | Sri Lankan rupee  | Colombo  | Southern Asia      | Asia   | Sri Lankan | 7.000000  | 81.000000  | 28120.00  | 21130.2  | 61860.0  |
| Sudan       | Republic of the Sudan                      | Sudanese pound    | Khartoum | Northern Africa    | Africa | Sudanese   | 15.000000 | 30.000000  | 694058.00 | 183596.0 | 1868000  |
| Thailand    | Kingdom of Thailand                        | Thai baht         | Bangkok  | South-Eastern Asia | Asia   | Thai       | 15.000000 | 100.000000 | 230100.00 | 198730.0 | 510890.0 |
| Ukraine     | Ukraine                                    | Ukrainian hryvnia | Kiev     | Eastern Europe     | Europe | Ukrainian  | 49.000000 | 32.000000  | 413110.00 | 96900.0  | 579400.0 |

dtype: int64

To display **5 random rows** from a Pandas DataFrame, use the `sample()` function